

Begründung für die Implementierung: KatS-Stab-Dashboard

1. Zielgruppenanalyse

Das Dashboard richtet sich an operative Einsatzkräfte und Stabsbereiche des Katastrophenschutzes (Feuerwehr, THW, Rettungsdienste). Diese arbeiten häufig unter Zeitdruck und starker kognitiver Belastung, weshalb die schnelle, gebündelte Erfassung einsatzkritischer Lageinformationen entscheidend ist. Deshalb werden verteilte Systemdaten auf einer einzigen, übersichtlichen Oberfläche zusammengeführt. Der Fokus liegt strikt auf Performance, Ausfallsicherheit und einer klaren Informationsarchitektur, um kritische Entscheidungen ohne Verzögerung zu ermöglichen.

2. Technische Randbedingungen

Eine zentrale Randbedingung ist der schonende Umgang mit den strikten Rate-Limits kostenfreier APIs (z. B. der Wetter-API, die bei dauerhaftem Abfragen dauerhaft blockiert). Um das System belastbar (resilient) zu machen, wurde eine Smart Polling-Architektur umgesetzt:

- **Idle-Sleep & Wakeup:** Das Backend überwacht permanent die Anzahl der verbundenen WebSocket-Clients. Sind keine Nutzer verbunden, pausiert das Polling vollständig ("Schlafmodus"), um die API-Kontingente zu schonen. Verbindet sich der erste Client, werden die Daten sofort aktualisiert.
- **Interne Cooldowns:** Service-spezifische Abklingzeiten (Wetter 10 Min., Pegel 5 Min., Einsatzdaten 1 Std.) liegen bewusst unter den Anbieterlimits, damit die zulässigen Request-Limits der Drittanbieter nicht überschritten werden.
- **Erkennung toter Verbindungen:** Verbindungen, die länger als 90 Sekunden keine Nachricht senden, werden automatisch geschlossen. Damit lösen abgehangene oder in den Ruhezustand versetzte Endgeräte keine unnötigen Abfragen mehr aus; ein Ping/Pong-Heartbeat hält lebende Verbindungen am Leben.
- **Transparenz des API-Budgets:** Open-Meteo limitiert die Gratis-API pro IP auf 10.000 "Aufrufe" pro Tag und berechnet den Verbrauch eines Requests nicht linear, sondern nach der Formel $(\text{Variablen}/10) \times (\text{Zeitraum in Tagen}/14)$. Unser Wetter-Request (23 Variablen, 8 Tage) kostet damit 1,31 Aufrufe. Das Backend zählt diesen Verbrauch fortlaufend (rollierend, exakt wie der Anbieter) und gibt ihn live im Wetter-Widget aus. So lässt sich die 10-Minuten-Aktualisierung belegen: 144 Abfragen pro Tag verbrauchen nur ca. 190 Aufrufe (rund 2 % des Kontingents) und lassen rund 9.800 Aufrufe für manuelle Refreshes frei.

3. Entscheidungen rund um die Datenquellen

Gemäß der Anforderung, realistische und valide Live-Daten abzubilden, wurden ausschließlich offizielle öffentliche Schnittstellen angebunden – statt auf Mock-Daten zurückzugreifen:

- **Wetter (Open-Meteo / DWD-ICON):** Die Daten des Deutschen Wetterdienstes liefern höchste lokale

Präzision für Berlin: aktuelle Werte, eine stündliche Prognose (24 Stunden) und die Tagesprognose der kommenden sieben Tage (ohne den heutigen Tag).

- Pegelstände (Pegelonline / WSV): Die REST-API der Wasserstraßen- und Schifffahrtsverwaltung wird genutzt, um Berliner Messstellen dynamisch abzufragen und 24-Stunden-Verläufe mit den Referenzwerten NNW bis HHW darzustellen. Die Tendenz (steigend/fallend) wird dabei nicht über eine Zwei-Punkt-Differenz, sondern als lineare Regression über einen wählbaren Zeitraum (1–24 h) berechnet, da die 15-Minuten-Messwerte um ± 2 cm schwanken und ein enges Vergleichsfenster sonst ständig das falsche Signal liefern würde.
- Einsatzdaten (Berliner Feuerwehr): Ein täglich aktualisierter CSV-Abzug aus dem Open-Data-Angebot liefert die Einsätze des Vortags. Für das farbliche Einsatzspektrum liefert eine zweite Datei (Hauptbeschwerden des Jahres) die Top-7-Meldebildkategorien, die auf die Rettungsdienstzahl des Vortags skaliert und als Treemap visualisiert werden.

Die Herkunft aller drei Quellen wird direkt an der jeweiligen Anzeige mit ihrer Lizenz (z. B. CC BY 4.0) kenntlich gemacht.

4. Entscheidungen für Technologien

Um Skalierbarkeit und leichte Wartbarkeit zu gewährleisten, wurde eine strikte Trennung von Frontend und Backend gewählt. Das systematische Abfragen läuft ausschließlich auf dem Server, was die Endgeräte der Einsatzkräfte entlastet.

- Backend (Python, FastAPI, Uvicorn, WebSockets): FastAPI und der Server Uvicorn bilden einen modernen Industriestandard für asynchrone Anwendungen. Über WebSockets werden neue Datenpakete per Push-Verfahren sofort an die Clients geschickt, statt jedes Gerät permanent per HTTP-Polling abfragen zu müssen. Zusätzlich kann das Backend das kompilierte Frontend selbst ausliefern – so benötigt die gehostete Variante nur einen einzigen Prozess.
- Frontend (React, Vite, Tailwind CSS, Recharts): React wurde wegen der komponentenbasierten Architektur und der hohen Marktdurchdringung gewählt. Das erleichtert das Onboarding neuer Entwickler und das Hinzufügen weiterer Fach-Widgets erheblich.

5. Details zur Benutzerschnittstelle

Die Oberfläche ist als Single-Page-Application aufgebaut und skaliert sich automatisch an die Bildschirmgröße an, sodass alle Fachbereiche auf einer Seite sichtbar bleiben. Ein umschaltbarer Hell- und Dunkelmodus deckt den Tagetrieb (maximale Kontraste) und schlecht beleuchtete Räume ab. Die Benutzeroberfläche ist interaktiv: Prognosedetails lassen sich aufklappen, die stündliche Vorschau über Pfeile durchblättern, und jedes Widget verfügt über einen manuellen Refresh-Button, der in kritischen Lagen eine gezielte Aktualisierung erlaubt. Beim Pegel-Widget lassen sich Messstellen zusätzlich suchen und anhand der Referenzwerte (z. B. „über MHW“) filtern, und der Trend-Zeitraum ist per Schalter zwischen 1, 3, 6, 12 und 24 Stunden wählbar (Standard 3 h). Im Wetter-Widget werden live der API-Verbrauch, die Kosten pro Anfrage und die Zeit bis zum täglichen Reset angezeigt. Bei einem Ausfall einer Schnittstelle fängt die Ansicht den Fehler ab und zeigt einen dezenten Hinweis an.